

0.84. Des vies et une fin de partie

Le poursuivant du 0.79, et 3 vies. Attrapé : une vie de moins, et tout le monde repart. À 0 : PERDU, et START pour recommencer.

1. Ce qu'il faut comprendre

C'est le poursuivant du 0.79, avec des vies. On en a 3. Chaque fois que le B nous attrape, on en perd une, et le A comme le B repartent de leur place de départ.

Ce qui est nouveau ici : l'ÉTAT du jeu. La variable `etat` dit où on en est : 0, on joue ; 1, on a perdu. Tout le programme est coupé en deux par `if (etat == 0) { ... } else { ... }` : en jeu, on bouge ; perdu, on attend.

Perdu : quand `vies` arrive à 0, `etat = 1;` et le mot PERDU s'écrit au milieu. Dans l'état 1, plus rien ne bouge.

Recommencer : dans l'état 1, le bouton **START** remet 3 vies et `etat = 0;` : on rejoue.

Repartir du départ après une prise : on efface la case (le A et le B y sont tous les deux), puis on remet `x`, `y`, `bx`, `by` à leurs valeurs de départ.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <un_pas>` → [0.29.4. La fonction un_pas\(\) — un seul pas, sans attendre](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1733 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          92 o ← la console, à l'allumage — la mise en route de la console
chaque_minuteur() 124 o ← main() — une fonction fournie par le langage
deplace_croix()    217 o ← main() — une fonction fournie par le langage
un_pas()           117 o ← main() — une fonction fournie par le langage
main()             272 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AttendreAcces      12 o ← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase        10 o ← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases        12 o ← effacer() — efface des cases
EcrireTexte        10 o ← texte() — écrit un texte, lettre par lettre
```

EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	704 o	← CopierTuiles	– la police entière
"VIES"	4 o	← texte()	– un texte gravé
"PERDU"	5 o	← texte()	– un texte gravé

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

effacer()	x 4	53 o sur place, dans deplace_croix(), un_pas(), main() + AdresseCase (10 o),
-----------	-----	--

EffacerCases (12 o)

poser()	x 2	26 o sur place, dans deplace_croix(), un_pas() + AdresseCase (10 o)
bouton()	x 5	25 o sur place, dans deplace_croix(), main() + LireManette (37 o)
texte()	x 2	22 o sur place, dans main() + EcrireTexte (10 o)
nombre()	x 1	10 o sur place, dans main() + EcrireNombre (33 o)
images()	x 2	4 o sur place, dans chaque_minuteur(), deplace_croix()
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

un_pas()	x 1	23 o sur place, dans main()
deplace_croix()	x 1	19 o sur place, dans main()
chaque_minuteur()	x 1	10 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- LES VARIABLES ----

#include <texte>           // écrit un texte à l'écran
#include <chaque>          // répond 1 toutes les n millisecondes
#include <un_pas>          // fait faire un seul pas à une tuile
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <deplace_croix>   // une tuile qui suit la croix, case par case
#include <effacer>         // efface des cases, ou tout le fond
#include <nombre>          // écrit un nombre en chiffres
#include <bouton>          // lit un bouton de la manette
```

2. UNE VARIABLE — X

```
uint8_t x = 10;           // le A : il part du milieu, (10, 8)
```

3. UNE VARIABLE — Y

```
uint8_t y = 8;
```

4. UNE VARIABLE — BX

```
uint8_t bx = 0;           // le B, le poursuivant : il part du coin, (0, 0)
```

5. UNE VARIABLE — BY

```
uint8_t by = 0;
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;           // la direction du prochain pas du B (le 0.79)
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;
```

8. UNE VARIABLE — VIES

```
uint8_t vies = 3;    // les vies qui restent : 3 au départ
```

9. UNE VARIABLE — ETAT

```
uint8_t etat = 0;    // L'ÉTAT DU JEU : 0 = on joue, 1 = perdu
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(0, 17, "VIES");           // le mot VIES, en bas à gauche

    while (true) {
        image();

        // ---- LE PROGRAMME EST COUPÉ EN DEUX PAR L'ÉTAT ----
        //
        //  if (etat == 0) { ... on joue ... } else { ... perdu : on attend START ... }
        //
        // Un seul des deux blocs se fait à chaque image, jamais les deux.
        if (etat == 0) {             // ===== ON JOUE

            // Le B avance vers le A, un pas tous les 600 ms (le 0.79).
            if (chaque(600)) {
                sx = 0;
                if (x > bx) { sx = 1; }           // le A est à droite : un pas à droite
                if (x < bx) { sx = -1; }          // à gauche : un pas à gauche
                sy = 0;
                if (y > by) { sy = 1; }           // plus bas : un pas en bas
                if (y < by) { sy = -1; }          // plus haut : un pas en haut
                if (sx != 0 || sy != 0) {         // pas déjà sur lui
                    un_pas(bx, by, ALPHABET[1], sx, sy);
                }
            }

            deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix

            // ATTRAPÉ : le B est sur la case du A.
            if (bx == x && by == y) {
                vies = vies - 1;                 // 1. une vie de moins : 3 → 2 → 1 → 0
                effacer(x, y, 1);                // 2. efface la case (le A et le B y sont)
                x = 10;                          // 3. tout le monde repart du départ :
                y = 8;                          //     le A au milieu...
                bx = 0;                          //     ...le B dans le coin
                by = 0;

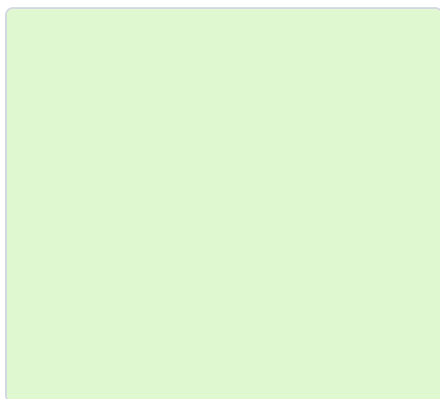
                if (vies == 0) {                 // 4. plus aucune vie ?
                    etat = 1;                   // on passe dans l'état « perdu »...
                    texte(7, 8, "PERDU");       // ...et on l'écrit au milieu
                }
            }

            nombre(5, 17, vies);                // les vies, après le mot VIES

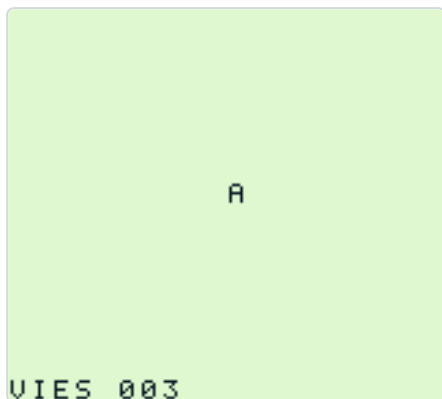
        } else {                               // ===== PERDU
            // Plus rien ne bouge : on attend seulement le bouton START.
            if (bouton(START)) {
                effacer(7, 8, 5);               // efface les 5 lettres de PERDU
                vies = 3;                       // de nouveau 3 vies...
                etat = 0;                       // ...et on rejoue
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

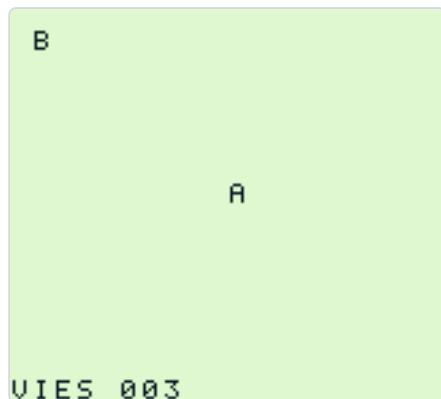
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



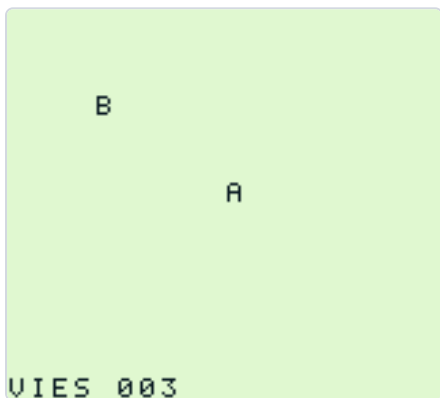
la toute première image — la console vient de démarrer — image 1



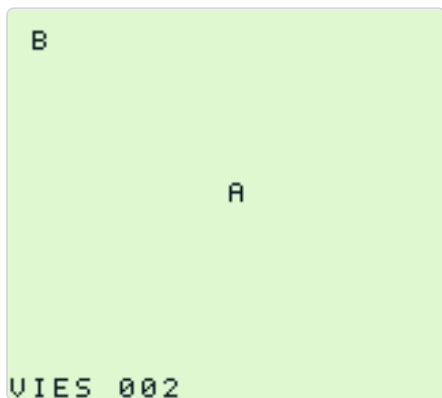
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



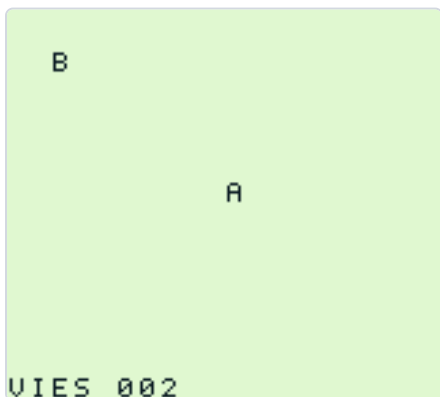
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

Le B poursuit le A ; à chaque prise, une vie de moins et tout repart. Au bout de trois : PERDU. START relance la partie.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ immobile, on perd les 3 vies : PERDU
 - ✓ START fait recommencer : 3 vies, en jeu
 - ✓ PERDU est effacé
-

Ensuite : **0.85. Plusieurs ennemis : un tableau de struct** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++